INTERVIEW PREPARATION — ROUND 2

Release Manager DevOps Expertise

Synechron — AI CoE Applications

Priority Q&A Bank, Grounded in Round 1 Interviewer Feedback

103 Questions & Answers · 22 Sections

Prepared September 15, 2026

Saroj Kumar Nayak

CONTENTS

A Opening / Leadership — 3 Q&A

B Release Management — 8 Q&A

C AWS — Very High Priority — 10 Q&A

D AWS Multi-Region — 3 Q&A

E Kubernetes Troubleshooting — 5 Q&A

F CI/CD — 4 Q&A

G Terraform — 3 Q&A

H SRE — 4 Q&A

I Production Incident Scenarios — 4 Q&A

J Release Governance — 4 Q&A

K Leadership / Coordination — 4 Q&A

L Metrics / Improvement — 4 Q&A

M Difficult Questions Because of Round 1 — 3 Q&A

N Very Likely Final Question — 1 Q&A

O Kubernetes Networking & Scaling Deep Dive — 8 Q&A

P AWS Infrastructure Deep Dive — 8 Q&A

Q CI/CD & GitOps Additional — 5 Q&A

R Terraform Additional — 4 Q&A

S SRE — Error Budgets, On-Call & Postmortems — 5 Q&A

T Additional Incident & Troubleshooting Scenarios — 6 Q&A

U Banking / Regulated Governance Additions — 4 Q&A

V Leadership — Client & Stakeholder (Consulting Context) — 3 Q&A

A. Opening / Leadership

Q1. Tell me about yourself.

I have 11+ years of experience across SRE, DevOps, cloud, CI/CD, Kubernetes and production operations. I started with middleware administration and gradually moved into DevOps and SRE. In my recent Microsoft Synapse project, I handled P1/P2 incidents, customer issues, SLI/SLO monitoring, observability and alert optimisation. I have also worked with Jenkins, Git, Terraform, Ansible, Rundeck and Kubernetes. My strength is combining hands-on engineering with production reliability, automation and release coordination.

Q2. Why are you interested in this role?

This role is a good match because it combines the areas I have worked on—SRE, DevOps, production operations, CI/CD and release coordination. I particularly like roles where I can improve deployment reliability, automate manual activities, coordinate production releases and use observability and SRE practices to reduce operational risk.

Q3. What do you understand about this role?

I see it as a combination of Release Management, DevOps and SRE. The responsibility is to ensure releases are technically ready, governance approvals are completed, dependencies are addressed, deployment is executed safely, production is validated and support handover is completed. At the same time, the role should continuously improve CI/CD automation and reduce deployment failures and manual toil.

B. Release Management

☐Q4. **Explain your end-to-end release process.**

I start with release scope, timeline and dependencies. Then I confirm branch and version strategy, build validation and DEV/SIT/UAT/PTU readiness. Before production, I verify testing sign-offs, ServiceNow change approval, implementation and rollback plans, monitoring and support readiness. Then I conduct the go/no-go review, coordinate the production deployment, perform smoke testing and monitoring validation, and finally complete operational handover, change closure and post-release review.

☐Q5. **What is your production readiness checklist?**

I check application build and configuration, infrastructure capacity, network connectivity, database and API dependencies, access, certificates, secrets, monitoring, alerts, logging, dashboards, backup and recovery requirements. I also verify the approved ServiceNow change, implementation plan, rollback plan, support availability, runbooks and stakeholder approvals.

☐Q6. **What happens during a go/no-go meeting?**

I review the release readiness criteria: testing and UAT sign-off, approved change, build validation, dependencies, production readiness, implementation and rollback plans, monitoring, support availability and open risks. Based on those inputs, stakeholders make the go/no-go decision. If a critical gate is incomplete, I recommend postponement or follow the approved exception process.

☐Q7. **What would you do if UAT approval is missing but business wants deployment?**

I would not bypass a mandatory governance gate. I would identify why the approval is pending, escalate to the appropriate business owner and obtain the required approval or an authorised exception according to the process. The decision and risk ownership should be documented.

☐Q8. **How do you handle dependencies?**

I maintain a dependency tracker with the dependency, owner, required date, status, risk and escalation path. I identify critical-path dependencies early and follow up before the release window. Any unresolved blocker is escalated during the readiness review rather than discovered during deployment.

☐Q9. **What is a rollback plan?**

A rollback plan defines exactly when and how we return to the previous stable version. It includes rollback triggers, commands or pipeline steps, database considerations, validation steps and ownership. I prefer having rollback criteria agreed before deployment rather than making that decision during an incident.

☐Q10. **What is blue-green deployment?**

Blue is the current production version and green is the new version. I deploy and validate green without immediately sending all traffic to it. Once validation succeeds, traffic is shifted to green. If there is an issue, traffic can be shifted back to blue, providing fast recovery.

☐Q11. **Blue-green vs canary?**

Blue-green normally maintains two environments and switches traffic between them, while canary gradually sends a small percentage of traffic to the new version. Canary is useful when we want to observe real production behaviour before increasing traffic progressively.

C. AWS — Very High Priority

☐Q12.**Explain an EKS architecture.**

An EKS architecture has the AWS-managed control plane and worker capacity running in multiple Availability Zones. Worker nodes can be managed node groups or self-managed nodes, and workloads run as Kubernetes pods. The cluster connects with VPC networking, load balancers, IAM, security groups, storage and other AWS services such as RDS. For production I would use private subnets for worker nodes, multiple AZs, appropriate ingress/load balancing, autoscaling and strong observability.

☐Q13.**How do pods get IP addresses in EKS?**

With the AWS VPC CNI, pods receive IP addresses from the VPC subnet, typically through ENIs attached to worker nodes. Therefore, subnet sizing is important because insufficient subnet IPs can prevent new pods or nodes from being scheduled.

☐Q14.**How do you calculate CIDR?**

For IPv4, the number of addresses is 2^(32-prefix). For example, a /24 provides 256 total addresses, while /16 provides 65,536 total addresses. In AWS, some addresses are reserved, so usable IPs are lower than the theoretical number. I size subnets based on expected nodes, pods, load balancers and future growth rather than selecting a CIDR arbitrarily.

☐Q15.**If you have 10 node groups, how do you decide the CIDR?**

I wouldn't select the CIDR simply based on the number of node groups. I first calculate expected nodes, pods per node, AWS VPC CNI IP requirements, ENI limits, load balancers and growth. Then I allocate sufficiently large VPC and subnet CIDRs across multiple AZs, leaving expansion capacity.

☐Q16.**How does EKS communicate with RDS?**

Pods communicate with RDS through the VPC network using the RDS endpoint. Security groups and network ACLs control access, and the RDS port —for example 5432 for PostgreSQL or 3306 for MySQL—must be permitted from the appropriate workload security group or network path. RDS should not be exposed publicly unless there is a specific requirement.

☐Q17.**How does traffic enter an EKS cluster?**

Typically external traffic reaches an AWS load balancer created through a Kubernetes Service of type LoadBalancer or an ingress controller. The ingress layer provides routing rules for HTTP/HTTPS traffic to Kubernetes services, which then route traffic to pods.

☐Q18.**What is an Ingress Controller?**

An Ingress Controller implements Kubernetes Ingress rules. It provides HTTP/HTTPS routing from external traffic to Kubernetes services and can handle TLS termination, host/path-based routing and other traffic-management functions.

☐Q19.**What is a Service Mesh?**

A service mesh manages service-to-service communication inside a distributed application. Technologies such as Istio can provide mTLS, traffic routing, retries, circuit breaking, telemetry and policy enforcement without requiring every application to implement these capabilities itself. I haven't owned a service mesh implementation directly, but I understand its architecture and operational purpose.

☐Q20.**How would you allow only five microservices to communicate with each other?**

I would use Kubernetes NetworkPolicies. I would define ingress and egress policies using pod labels, namespaces and ports. By default, I would establish a deny policy and then explicitly allow only the required communication paths.

☐Q21.**How would you troubleshoot a NetworkPolicy issue?**

I first verify pod labels and namespaces, then inspect the NetworkPolicy selectors and allowed ports. I check whether ingress or egress is being denied and test connectivity from the source pod. I also verify the CNI implementation and relevant security groups or network controls if traffic leaves the cluster.

D. AWS Multi-Region

☐ Q22. **How would you design active-active architecture across two AWS regions?**

I would deploy equivalent application stacks in both regions, preferably across multiple AZs. Global traffic management would direct users to healthy regional endpoints. Data would require an appropriate multi-region replication strategy depending on the database. I would also ensure consistent infrastructure through Terraform, centralised observability and tested failover procedures.

☐ Q23. **Route 53 vs AWS Global Accelerator?**

Route 53 is DNS-based traffic management and can use health checks and routing policies such as failover or latency routing. Global Accelerator operates at the network layer using static anycast IPs and can route traffic to healthy regional endpoints. For latency-sensitive active-active applications where I want fast regional failover without relying solely on DNS propagation, Global Accelerator can be a strong option.

☐ Q24. **How does Global Accelerator detect a failed region?**

Global Accelerator uses health checks against configured endpoints. If an endpoint becomes unhealthy, traffic is directed to another healthy endpoint according to the configured traffic-routing policy.

E. Kubernetes Troubleshooting

☐ Q25. **A pod is in CrashLoopBackOff. What do you do?**

I first check the pod status and events, then inspect logs, including previous-container logs if it is restarting. I check environment variables, ConfigMaps, Secrets, probes, resource limits and recent deployments or configuration changes. Once I identify the cause, I mitigate it, validate the pod becomes Ready and then perform RCA.

Commands:
kubectl get pods
kubectl describe pod <pod>
kubectl logs <pod>
kubectl logs <pod> --previous
kubectl get events --sort-by=.lastTimestamp

☐ Q26. **Pod is Pending. What do you check?**

I check pod events first. Then I verify node capacity, CPU/memory requests, taints and tolerations, node selectors, affinity rules, resource quotas and available IP capacity. In EKS I also check subnet IP availability because VPC CNI IP exhaustion can prevent pod placement.

☐ Q27. **ImagePullBackOff?**

I check the pod events for the exact image-pull error. Then I validate image name and tag, registry availability, imagePullSecrets or ECR permissions, node IAM permissions and network connectivity to the registry.

☐ Q28. **OOMKilled?**

I check container memory usage and configured requests and limits. I compare actual usage against the limit, inspect application logs and identify whether there is a memory leak or an unexpected workload increase. I would mitigate by adjusting resources if justified, but also fix the underlying application issue.

☐ Q29. **Deployment is stuck during rollout. What do you check?**

I check rollout status, replica availability, pod events, readiness probes, image availability, resource constraints and application logs. I compare the new ReplicaSet with the previous version and inspect recent configuration changes. If the release is impacting production and rollback criteria are met, I rollback to the last stable version.

kubectl rollout status deployment/<name>
kubectl get rs
kubectl get pods
kubectl describe pod <pod>
kubectl rollout history deployment/<name>
kubectl rollout undo deployment/<name>

F. CI/CD

☐Q30.**Design a production CI/CD pipeline.**

I would design it as:

Git → Build → Unit Test → Code Quality/SAST → Dependency Scan → Build Image → Container Scan → Push Artifact → DEV → SIT → Automated Tests → UAT Approval → PTU → Change Approval → Production → Smoke Test → Monitoring

I would build the artifact once and promote the same immutable artifact through environments.

☐Q31.**How do you make a pipeline secure?**

I use protected branches, pull-request reviews, least-privilege service accounts, secret management, SAST, dependency scanning, container scanning and approval gates. Production credentials should never be hard-coded in pipelines. Access should be controlled using managed identities, service connections or equivalent secure mechanisms.

☐Q32.**Jenkins vs GitHub Actions?**

Both can implement CI/CD. Jenkins provides a highly extensible self-managed automation platform with a large plugin ecosystem. GitHub Actions is tightly integrated with GitHub repositories and provides managed runners and workflow automation. The choice depends on the organisation's existing ecosystem, security model and operational requirements.

☐Q33.**How do you manage secrets in CI/CD?**

I don't store secrets directly in Git or pipeline YAML. I use a secret-management solution such as Azure Key Vault, AWS Secrets Manager or another approved vault, with controlled access through service identities or workload identities.

G. Terraform

☐ Q34. **How do you use Terraform in production?**

I use Terraform to provision infrastructure consistently and repeatably. I keep code in Git, use remote state with locking, review changes through pull requests and run plan before apply. For production, apply should have appropriate approval and change governance.

☐ Q35. **What is Terraform state?**

Terraform state maintains the mapping between the configuration and the real infrastructure resources. It allows Terraform to determine what needs to be created, modified or destroyed. In production I store state remotely with access control and locking.

☐ Q36. **What happens if Terraform state is lost?**

Terraform loses its knowledge of the existing resource mapping. I would first recover the remote state from backup if available. If necessary, existing resources can be imported into Terraform state. I would avoid blindly running apply because that could create duplicate or destructive changes.

H. SRE

☐Q37.**What is SLI, SLO and SLA?**

SLI is the measured reliability indicator, such as availability or latency. SLO is the internal reliability target, such as 99.9% availability. SLA is the external/business commitment, usually with consequences if the agreed target isn't met.

☐Q38.**What are the four Golden Signals?**

Latency, Traffic, Errors and Saturation.

☐Q39.**How do you reduce alert noise?**

I analyse alerts based on frequency, severity, actionability and correlation with actual incidents. I remove duplicate or non-actionable alerts, tune thresholds, introduce grouping and routing and create alerts around symptoms that require action. In my recent project, I worked on alert optimisation and reduced alert noise significantly.

☐Q40.**How do you measure reliability?**

I use SLIs and SLOs such as availability, latency, error rate and successful transactions. I also monitor incidents, MTTR, change failure rate and error-budget consumption to understand reliability from both technical and operational perspectives.

I. Production Incident Scenarios

☐ Q41. **A deployment causes 20% errors. What do you do?**

I first assess customer impact and stop further rollout. I check application errors, logs, latency, infrastructure metrics and recent changes. If the release is the likely cause and rollback criteria are met, I rollback to the stable version. Then I validate recovery through metrics and smoke tests, communicate the status, and later perform RCA and preventive actions.

☐ Q42. **Production deployment succeeded but application isn't responding.**

I would check the deployment and pod health first, then services, endpoints, ingress/load balancer, DNS, network policies and application logs. I would compare the new version with the previous healthy version. If the deployment introduced the problem, I would rollback while continuing investigation.

☐ Q43. **CPU suddenly reaches 95%.**

I first assess whether there is customer impact. Then I identify the affected workloads and correlate CPU with traffic, deployment changes and application behaviour. I check pod/node metrics and logs. If appropriate, I scale the workload or infrastructure to mitigate the impact, then identify the underlying cause and prevent recurrence.

☐ Q44. **A critical production deployment fails halfway through. What do you do?**

I pause the deployment and establish the exact state of the environment. I identify which components were successfully deployed and which were not. Based on the predefined rollback strategy, I either rollback to the last known-good state or safely complete the deployment. After stabilisation, I validate the complete application flow and document the incident.

J. Release Governance

☐Q45. **What should a release note contain?**

Release version, scope, features/fixes, known issues, dependencies, affected components, deployment window, prerequisites, validation steps, risks and support information.

☐Q46. **What should an implementation plan contain?**

Detailed deployment steps in execution order, prerequisites, owners, timings, commands or pipeline references, validation steps, dependencies, communication points and rollback instructions.

☐Q47. **What is audit evidence?**

Audit evidence demonstrates that the required governance controls were actually completed. Examples include approved ServiceNow changes, testing evidence, UAT sign-off, deployment logs, approval records, release notes, implementation and rollback plans and post-release validation.

☐Q48. **Why is CMDB important?**

CMDB provides visibility into configuration items and their relationships. During releases it helps identify affected components, dependencies and ownership and provides traceability for changes.

K. Leadership / Coordination

☐Q49. **How do you handle conflict between Development and Production Support?**

I focus on facts and production impact rather than opinions. I establish the issue, evidence, risk and options, then bring the relevant teams together to agree on the safest action. If there's disagreement around a production release, I escalate through the defined decision-making process and document the decision.

☐Q50. **How do you handle a developer who says, “The application works in my environment”?**

I would acknowledge that the application may work in their environment, but production readiness requires validation across configuration, infrastructure, dependencies, security, monitoring and operational support. I would use environment-specific evidence and testing results to identify the difference rather than making it a personal disagreement.

☐Q51. **How do you manage multiple releases at the same time?**

I prioritise based on business criticality, production risk, dependencies and release windows. I maintain a release calendar and dependency tracker, assign clear owners and establish readiness checkpoints. Critical conflicts are escalated early so that we don't create competing production changes.

☐Q52. **How do you communicate during a critical release?**

I keep communication concise and factual: current status, completed activity, next activity, issue or risk, customer impact if any, owner and expected next update. During a war room I maintain one source of truth so different teams don't receive conflicting information.

L. Metrics / Improvement

☐Q53. **What KPIs would you use for release management?**

Release success rate, change failure rate, rollback rate, deployment duration, release delays, production defects, post-release incidents, emergency changes, manual deployment effort and MTTR.

☐Q54. **How would you improve the release process?**

I would first analyse historical release failures and delays. Then I would standardise checklists, automate repetitive activities, introduce pipeline quality gates, improve environment validation, strengthen observability and make rollback easier. Finally, I would measure the impact through release KPIs and continuously improve.

☐Q55. **What is toil?**

Toil is repetitive, manual, predictable operational work that provides little lasting value and can often be automated.

☐Q56. **Give an example of reducing toil.**

In my previous environment, some post-deployment activities were being performed manually. I automated those activities through Rundeck and API-based execution, so the team could trigger the process consistently instead of logging into systems and performing the same steps manually.

M. Difficult Questions Because of Round 1

☐ Q57. **You couldn't explain service mesh in the first round. What do you know about it?**

I wasn't able to explain it clearly in the first discussion. Conceptually, a service mesh handles service-to-service communication using a dedicated infrastructure layer. It provides capabilities such as mTLS, traffic routing, retries, circuit breaking, observability and policy enforcement. I haven't owned a service mesh implementation directly, so I would be transparent about that while explaining the architecture.

☐ Q58. **You were unclear about EKS CIDR. Explain it now.**

CIDR determines the IP address capacity of the network. For IPv4, a /24 has 256 total addresses and a /16 has 65,536. For EKS, I don't size it simply based on node count. I calculate node requirements, pod density, VPC CNI IP consumption, load balancers and future growth, then allocate subnet CIDRs across multiple AZs with sufficient headroom.

☐ Q59. **What did you learn from your previous production issues?**

One key lesson is that production reliability depends not only on application health but also on dependencies, capacity, observability and operational readiness. I therefore try to validate dependencies and monitoring before deployment and automate repetitive checks wherever possible.

N. Very Likely Final Question

☐Q60. **Why should we hire you?**

I bring a combination of 11+ years of production engineering experience, SRE, DevOps, cloud, Kubernetes, CI/CD and automation. I understand both the technical side of deployments and the operational side—risk, governance, monitoring, incident management and production support. I can work across Engineering, QA, Infrastructure and Support teams while still being hands-on technically. My focus would be to make releases more reliable, automated, traceable and predictable.

O. Kubernetes Networking & Scaling Deep Dive

☐ Q61. **What is CoreDNS and how does service discovery work in Kubernetes?**

CoreDNS runs as pods in kube-system and provides internal DNS resolution for services and pods. Each Service gets a DNS name like servicename.namespace.svc.cluster.local, and CoreDNS resolves it to the Service's ClusterIP, which kube-proxy then load-balances across pod endpoints. If DNS lookups start failing, I check CoreDNS pod health first, then look at its ConfigMap for any recent changes.

☐ Q62. **What are the different Kubernetes Service types, and when do you use each?**

ClusterIP is the default, internal-only, used for service-to-service communication. NodePort exposes a port on every node, mostly used for testing or when there's no external load balancer available. LoadBalancer provisions a cloud load balancer, used for external-facing production services. Headless Service (clusterIP: None) skips load-balancing and returns individual pod IPs directly, which is what StatefulSets use for stable per-pod identity.

☐ Q63. **iptables vs IPVS mode in kube-proxy — what's the difference and why does it matter at scale?**

kube-proxy in iptables mode implements Service routing using iptables rules, which works fine at smaller scale but rule evaluation is sequential, so latency grows roughly with the number of Services. IPVS mode uses a hash-table-based load balancer in the kernel, so lookup is near-constant time regardless of Service count. For large clusters with thousands of Services, IPVS is the better choice for predictable performance.

☐ Q64. **Deployment vs StatefulSet — when do you use each?**

Deployment is for stateless workloads where pods are interchangeable — any pod can serve any request, and pods get random names. StatefulSet is for workloads needing stable network identity and storage — each pod gets a fixed name and its own PersistentVolumeClaim that survives rescheduling, which is what databases and other stateful systems need.

☐ Q65. **What is a PodDisruptionBudget and why does it matter during cluster maintenance?**

A PodDisruptionBudget defines the minimum number or percentage of pods that must stay available during voluntary disruptions, like a node drain for maintenance or a cluster upgrade. Without one, a node drain could take down every replica of a service at once. I'd set a PDB on production-critical Deployments so Kubernetes blocks a drain that would violate availability.

☐ Q66. **Explain how HPA, VPA, and Cluster Autoscaler are different and how they work together.**

HPA scales the number of pod replicas based on metrics like CPU or custom metrics. VPA adjusts a pod's resource requests/limits over time based on observed usage, rather than changing replica count. Cluster Autoscaler adds or removes nodes when pods can't be scheduled due to insufficient capacity, or when nodes are underutilized. HPA and Cluster Autoscaler are commonly used together — HPA scales pods, and if there's no room, Cluster Autoscaler adds nodes to fit them; VPA is used more carefully since it can restart pods to apply new values.

☐ Q67. **What is IRSA (IAM Roles for Service Accounts) in EKS, and why is it important?**

IRSA lets a Kubernetes Service Account assume a specific AWS IAM role via OIDC federation, so a pod gets exactly the AWS permissions it needs instead of inheriting the broader node IAM role. This follows least privilege — without IRSA, every pod on a node effectively shares that node's IAM permissions, which is a real security risk in a multi-tenant cluster. It's something I'd specifically check for in a security review of an EKS environment.

☐ Q68. **What's the difference between EKS managed node groups, self-managed nodes, and Fargate?**

Managed node groups let AWS handle node provisioning and lifecycle — patching, scaling, and graceful draining are largely automated. Self-managed nodes give full control over the AMI and configuration but the operational burden is entirely on the team. Fargate removes node management entirely — pods run on AWS-managed compute with no visible nodes, good for variable or bursty workloads but with some limitations around DaemonSets and certain networking patterns.

P. AWS Infrastructure Deep Dive

☐Q69. **How do you approach EKS cluster version upgrades safely?**

I check the deprecation and version-skew policy first — control plane and node versions can only be a limited number of minor versions apart. I upgrade the control plane first, then update EKS add-ons (VPC CNI, CoreDNS, kube-proxy) to compatible versions, then roll node groups gradually rather than all at once, validating workloads after each step. I always test the upgrade path in a lower environment first and have a rollback/redeploy plan ready.

☐Q70. **What are EKS add-ons and why do their versions matter?**

Add-ons like the VPC CNI, CoreDNS, and kube-proxy are core cluster components that AWS can manage directly. Their versions need to stay compatible with the EKS control-plane version — running a mismatched CNI version after a cluster upgrade is a common cause of networking issues post-upgrade, so I always check the compatibility matrix before and after an upgrade.

☐Q71. **NAT Gateway vs NAT instance — which would you choose and why?**

NAT Gateway is AWS-managed, highly available within an AZ, and scales automatically, but costs more and charges per GB processed. A NAT instance is self-managed, cheaper at small scale but requires patching and doesn't scale automatically. For production I'd default to NAT Gateway for reliability, one per AZ for high availability, unless there's a strong cost-driven reason to self-manage.

☐Q72. **VPC Peering vs Transit Gateway vs PrivateLink — when do you use each?**

VPC Peering is a direct one-to-one connection, simple but doesn't scale well past a handful of VPCs since it's not transitive. Transit Gateway acts as a central hub connecting many VPCs and on-prem networks, better for larger or hub-and-spoke topologies. PrivateLink exposes a specific service privately without exposing the whole network — the right choice when you only need access to one service, like a shared internal API, without full network connectivity.

☐Q73. **How does the AWS Load Balancer Controller work with EKS Ingress?**

The AWS Load Balancer Controller watches Kubernetes Ingress and Service resources and provisions actual ALBs or NLBs in AWS to match them, using annotations to control things like scheme, target type, and health-check settings. It's the standard way to get proper Layer 7 routing and TLS termination for EKS workloads instead of relying on the basic Service type LoadBalancer, which only gives you an NLB with no routing rules.

☐Q74. **Secrets Manager vs Parameter Store vs Kubernetes Secrets — how do you decide?**

Kubernetes Secrets are base64-encoded, not encrypted by default at rest unless envelope encryption is enabled, and are cluster-scoped — fine for simple cases but not a strong secrets story on their own. AWS Secrets Manager adds automatic rotation and is the better choice for credentials like database passwords that need rotation. Parameter Store is cheaper and good for configuration values that don't need rotation. In EKS, I'd typically sync from Secrets Manager or Parameter Store into the cluster using something like the AWS Secrets and Configuration Provider, rather than storing sensitive values directly as native Kubernetes Secrets.

☐Q75. **How would you troubleshoot intermittent connection timeouts between an EKS pod and an external API?**

I'd isolate whether it's DNS, network path, or the external service itself — test DNS resolution from within the pod, then test raw connectivity with curl or telnet to the endpoint and port. I'd check security groups and NACLs along the path, NAT Gateway capacity and port-allocation errors if it's high-volume outbound traffic, and whether the issue correlates with pod restarts, node changes, or scaling events. If it's intermittent rather than constant, that pattern usually points to NAT port exhaustion or a flapping load-balancer health check rather than a static misconfiguration.

☐Q76. **What's the difference between a Security Group and a Network ACL, and when would you use both together?**

Security Groups are stateful and applied at the instance/ENI level — if you allow inbound traffic, the response is automatically allowed out. NACLs are stateless and applied at the subnet level, so both inbound and outbound rules need to be defined explicitly, and they're evaluated in rule-number order. I use Security Groups as the primary control since they're easier to reason about, and add NACLs as a coarser subnet-level boundary, particularly useful for explicit deny rules that Security Groups can't express.

Q. CI/CD & GitOps Additional

☐Q77. **What is GitOps, and how does it differ from a traditional push-based CI/CD pipeline?**

In traditional CI/CD, the pipeline pushes changes directly into the target environment. In GitOps, a controller like ArgoCD or Flux running inside the cluster continuously pulls the desired state from a Git repository and reconciles the live environment to match it. This means Git is the single source of truth for what's actually running, drift gets automatically corrected or flagged, and rollback is just reverting a Git commit.

☐Q78. **How would you implement automated rollback based on metrics after a canary release?**

I'd use a progressive-delivery tool like Argo Rollouts or Flagger, which shifts traffic incrementally to the new version while querying a metrics provider like Prometheus for error rate and latency at each step. If the metrics breach a defined threshold at any step, it automatically halts and rolls back traffic to the stable version without waiting for a human to notice — a meaningful improvement over a manual canary where someone has to be watching a dashboard.

☐Q79. **What's your view on trunk-based development vs GitFlow for a team practicing continuous deployment?**

Trunk-based development, with short-lived branches and feature flags to hide incomplete work, fits well with high-frequency continuous deployment because it avoids long-lived branches that drift and become painful to merge. GitFlow's longer release branches suit lower-frequency, more heavily gated releases where you want a stable, freezable artifact. The choice should follow the team's actual release cadence and governance needs, not be picked by default.

☐Q80. **What's the use case for Rundeck in a DevOps/SRE toolchain?**

Rundeck is useful for turning ad hoc scripts and manual runbook steps into a self-service, access-controlled, auditable job — instead of someone SSH-ing into a box and running a command by hand, they trigger a defined job through Rundeck with proper logging and role-based access. I've used it to give support teams a safe, controlled way to run specific operational tasks without needing full production access.

☐Q81. **How do you manage Helm chart versioning and values across environments?**

I keep the chart itself environment-agnostic and use separate values files per environment (values-dev.yaml, values-prod.yaml) layered on top of a base values.yaml, so environment differences are explicit and reviewable in Git. Chart versions are bumped deliberately and tagged, and I promote the same chart version across environments rather than letting each environment drift to a different chart version over time.

R. Terraform Additional

☐ Q82. **How do you structure Terraform for multiple environments and teams?**

I use separate state files per environment, typically through workspaces or separate backend configurations, so a mistake in dev can't touch production state. Reusable logic goes into modules, versioned and referenced by tag, so environments consume the same tested module rather than duplicating code. For multi-team setups, I'd also separate state by ownership boundary, not just by environment, so one team's apply can't accidentally affect another team's resources.

☐ Q83. **What causes Terraform drift, and how do you detect and handle it?**

Drift happens when the real infrastructure changes outside of Terraform — a manual console change, another tool, or an emergency fix. I detect it by running terraform plan regularly, including in a scheduled CI job, and treating any unexpected diff as something to investigate before it's forgotten. Handling it means either updating the Terraform code to match the intentional change, or reverting the manual change so infrastructure matches code again — never just re-running apply blindly without understanding the diff.

☐ Q84. **What's the risk of using terraform apply -target, and when is it justified?**

-target applies changes to only a specific resource, bypassing Terraform's normal dependency-graph evaluation for the rest of the configuration. The risk is it can leave state and real infrastructure inconsistent with what a full plan would produce, and repeated use tends to mask real configuration problems. I'd only use it for genuine emergency situations — like needing to fix one broken resource fast — and always follow up with a full plan/apply to reconcile everything afterward.

☐ Q85. **How does remote state locking work, and why is it important with a team?**

Remote state locking — for example DynamoDB locking alongside an S3 backend — prevents two people or two pipeline runs from applying against the same state simultaneously, which could corrupt state or produce conflicting changes. Without locking, a race condition between two applies can leave the state file inconsistent with either intended change.

S. SRE — Error Budgets, On-Call & Postmortems

☐ Q86. **What is an error budget, and how would you use it to make a release decision?**

An error budget is the allowed amount of unreliability under an SLO — if the SLO is 99.9% availability, the error budget is the remaining 0.1%. If the error budget is nearly exhausted, that's a signal to slow down releases and prioritize reliability work over new features; if budget remains, there's room to take on more release risk. It turns “are we reliable enough” into a concrete, pre-agreed number rather than a subjective argument.

☐ Q87. **What's the difference between a burn-rate alert and a static threshold alert?**

A static threshold alert fires when a metric crosses a fixed value, regardless of how fast the error budget is actually being consumed. A burn-rate alert looks at how quickly the error budget is being used relative to the SLO window — a fast burn over a short window triggers a page because the budget will be exhausted soon, while a slow burn over a long window can be a lower-urgency ticket. This gives faster, more accurate signal for genuinely urgent problems versus ones that can wait.

☐ Q88. **How would you structure an on-call rotation and escalation policy?**

I'd define a primary and secondary on-call per rotation, with a clear handoff process and documented context at shift change. Escalation policy should specify how long the primary has to acknowledge before it pages the secondary, and a further escalation path to a lead or manager for anything unresolved past a defined SLA. Runbooks for common alerts should be linked directly from the alert itself, so on-call isn't starting from zero at 2am.

☐ Q89. **What does a blameless postmortem process look like, and why does the “blameless” part matter?**

A blameless postmortem focuses on what happened, what conditions allowed it to happen, and what systemic changes prevent recurrence — not on who made the mistake. It matters because if people fear blame, they hide information or downplay their role, and you never get the full picture of what actually went wrong. The output should be specific, owned action items, not just a narrative of the incident.

☐ Q90. **How do you decide what should page someone at night versus wait for business hours?**

Only symptoms with real, current or imminent customer impact should page — a burn-rate alert on a customer-facing SLO, a full outage, a security incident. Anything degraded but not yet customer-impacting, or a slow-trending issue with time to respond during business hours, should go to a ticket queue instead. Getting this wrong in either direction is costly — over-paging causes fatigue and missed real alerts, under-paging means real problems sit unnoticed.

T. Additional Incident & Troubleshooting Scenarios

☐ Q91. **A certificate expires during a production deployment window. What do you do?**

First I check the actual blast radius — is it a load balancer/TLS cert affecting live traffic, or an internal service cert. If it's customer-facing, I'd treat it as the priority over the planned deployment and either pause the release or fix the cert in parallel if it's unrelated. I'd renew and redeploy the cert through the same pipeline used to manage it originally, not a manual one-off fix, so it stays reproducible and doesn't drift from what's tracked in code.

☐ Q92. **A pod can't reach an RDS database after a fresh EKS deployment, but it worked in the previous cluster. What do you check?**

I'd check security groups first — does the new node/pod security group have access to the RDS security group on the DB port. Then check subnet routing — are the EKS nodes in subnets that can route to the RDS subnet, and is there a NAT Gateway if RDS is reached over a peering connection. I'd also check whether IRSA or a stored credential the pod needs is actually available in the new cluster, since that's a common gap when standing up a new environment.

☐ Q93. **A deployment succeeds, health checks pass, but customers report errors intermittently. What's your approach?**

This pattern often means only some pods or some AZs are affected, not all — so an aggregate health check passing can hide a partial failure. I'd check per-pod and per-AZ error rates rather than the aggregate, check for uneven traffic distribution, and check whether it correlates with a specific pod, node, or availability zone. If it's isolated to a subset, I'd cordon or replace the affected pods/nodes rather than assuming the whole release is bad.

☐ Q94. **How do you troubleshoot a Kubernetes DNS resolution failure inside a pod that only happens intermittently?**

Intermittent DNS failures in Kubernetes are frequently a CoreDNS scaling or conntrack issue — I'd check CoreDNS pod CPU/resource limits and whether it's being throttled, and check for the known UDP conntrack race condition under high query volume. I'd also check whether ndots settings in the pod's resolv.conf are causing unnecessary extra lookups, since Kubernetes' default ndots:5 can multiply DNS query volume for external domains.

☐ Q95. **Post-deployment, a service starts throwing IAM permission-denied errors it didn't have before. What do you check?**

I'd check whether the release changed the service account or the associated IAM role/policy — a common cause is a new API call the updated code makes that the existing role was never scoped for. I'd check IRSA mapping is still correctly bound to the right service account, and check CloudTrail for the specific denied action to identify exactly which permission is missing, rather than guessing and over-granting broad permissions to make the error go away.

☐ Q96. **A release passed all automated tests and smoke tests, but a downstream team reports broken integration two days later. How do you handle it?**

I'd first confirm the release is actually the cause — check timing correlation and what specifically changed in the interface or contract between the two services. If confirmed, I'd assess whether it's safe to roll back at this point, given two days of other changes may have layered on top, or whether a forward fix is now the safer path. Longer-term, this usually points to a gap in integration/contract testing between services that should be added rather than relying on smoke tests within one service alone.

U. Banking / Regulated Governance Additions

☐Q97. **In a regulated banking environment, why is segregation of duties important in the release process, and how is it typically enforced?**

Segregation of duties means the person who writes or approves a change shouldn't be the same person who deploys it to production unsupervised — it's a control against both error and fraud that banking regulators specifically look for. It's typically enforced through pipeline permissions (deploy access separate from approval rights), mandatory second-approver gates in the change record, and audit logs showing who did what, so no single person can push an unreviewed change to production.

☐Q98. **What's the difference between a control gap and a control failure in an audit context, and why does that distinction matter?**

A control gap means a required control doesn't exist or wasn't designed for a situation — a process weakness. A control failure means the control existed and was supposed to catch something, but didn't work as intended for a specific instance — an execution weakness. The distinction matters because the remediation is different: a gap needs a new control designed, a failure needs the existing control fixed or better enforced, and auditors typically expect you to know which one you're reporting.

☐Q99. **How would you handle a request to bypass change approval for a "trivial" production fix?**

I wouldn't bypass it, even for something that looks trivial — "trivial" changes are exactly the ones that sometimes turn out not to be, and the approval process exists precisely so that judgment isn't made unilaterally under time pressure. I'd use the expedited/emergency change path if genuine urgency exists, which still requires an approval, just a faster one, and make sure it's documented after the fact if verbal approval was needed to move fast.

☐Q100. **How do you make sure release documentation would hold up if a regulator asked for evidence six months later?**

I write it assuming the reader has zero memory of the day it happened — clear timestamps, named approvers, the exact artifact version, and explicit pass/fail results rather than vague statements like "testing was fine." I keep it in the system of record linked to the change ticket, not in chat messages or personal notes that might not be retained, since retention and traceability are usually what an audit actually checks first.

V. Leadership — Client & Stakeholder (Consulting Context)

☐Q101. **As a consulting-firm employee working inside a client's release process, how do you handle a disagreement with the client's own architecture/governance team?**

I'd present the technical concern clearly with evidence, and be explicit that the final call on their own standards is the client's to make, not mine to override. My job is to make sure they're deciding with full information — the risk, the alternatives, and the trade-offs — not to win the argument. If I still disagree after that, I'd document my recommendation and respect their decision, escalating only if I believe it creates a genuine compliance or safety risk.

☐Q102. **How do you build credibility quickly with a new client team you haven't worked with before?**

[Bring your own specifics here — this should reflect how you've actually done it.] A reasonable structure: come in listening more than prescribing in the first few weeks, understand their existing process and why it evolved that way before suggesting changes, and deliver a few early, reliable wins on smaller asks before pushing for bigger process changes. Credibility in a consulting context is usually earned through consistent follow-through on small things, not one impressive technical answer.

☐Q103. **How do you communicate a delay or problem to a client stakeholder without damaging trust?**

Early and factually — the trust damage usually comes from a stakeholder finding out later than they should have, not from the problem itself. I'd state what happened, the actual impact, what's being done about it, and when they'll hear the next update, and avoid over-promising a fix timeline I'm not confident in just to soften the message in the moment.

Quick Reference — Day-Of Cram Sheet

If you don't have time to review everything, focus on these 15 first.

☐Tell me about yourself

☐End-to-end release process

☐Production readiness checklist

☐Go/no-go

☐Rollback

☐CI/CD pipeline design

☐EKS architecture

☐EKS CIDR/IP sizing

☐EKS → RDS connectivity

☐Ingress vs Service Mesh

☐NetworkPolicy

☐Multi-region active-active

☐Route 53 vs Global Accelerator

☐CrashLoopBackOff / Pending / ImagePullBackOff

☐Production deployment failure

EKS NETWORKING

```
VPC → Subnets → ENI → Node → Pod
```

TRAFFIC

```
Client → Load Balancer → Ingress → Service → Pod
```

RELEASE

```
Git → Build → Test → Artifact → DEV → SIT → UAT → PTU → Approval → Prod → Smoke Test → Monitor → Handover
```

INCIDENT

```
Assess → Triage → Mitigate → Fix/Rollback → Validate → Communicate → RCA → Prevent
```

SRE

```
SLI → SLO → Error Budget → Observability → Reliability
```

> *The biggest improvement from Round 1 to Round 2: don't guess when you don't know something. Give a clean conceptual answer, explicitly separate hands-on experience from conceptual knowledge, and move on confidently.*